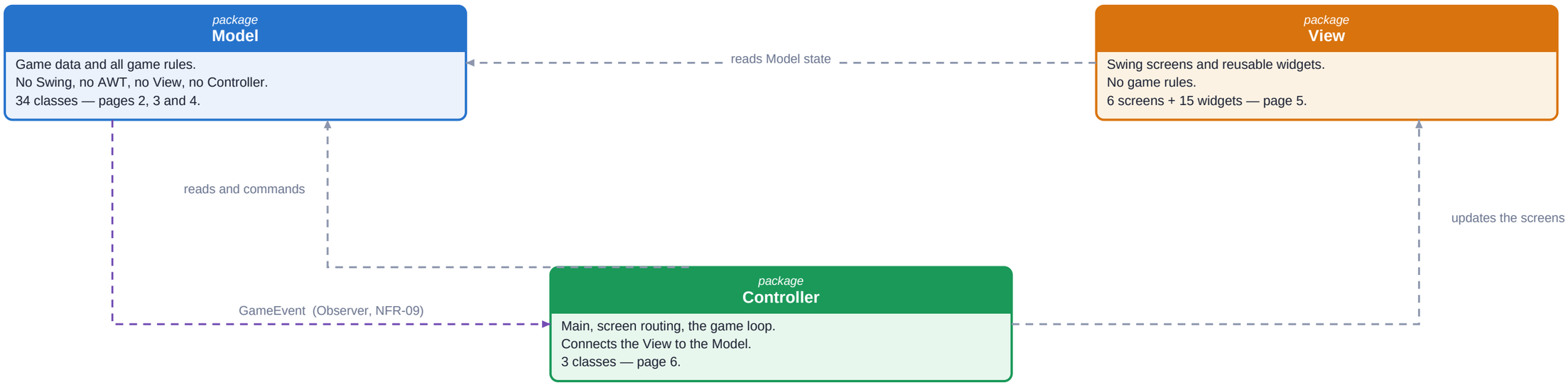
Educational Ludo — Sharks

Class diagram of the system · Iteration 1 implemented, Iterations 2–3 planned

Owner: Mohamed Zereny · Reviewer: Rakan Ziyan · SRS: NFR-08 (MVC), NFR-09 (design patterns), NFR-20 (packages)

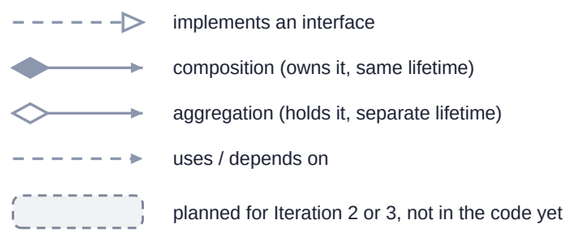
Every class shown as implemented was read off Iteration1/src at iteration-1 commit 3d3ce9b. Editable source: Iteration1/docs/CLASS_DIAGRAM.md



The dependency rule (NFR-08, NFR-20) is enforced by a test, not only documented

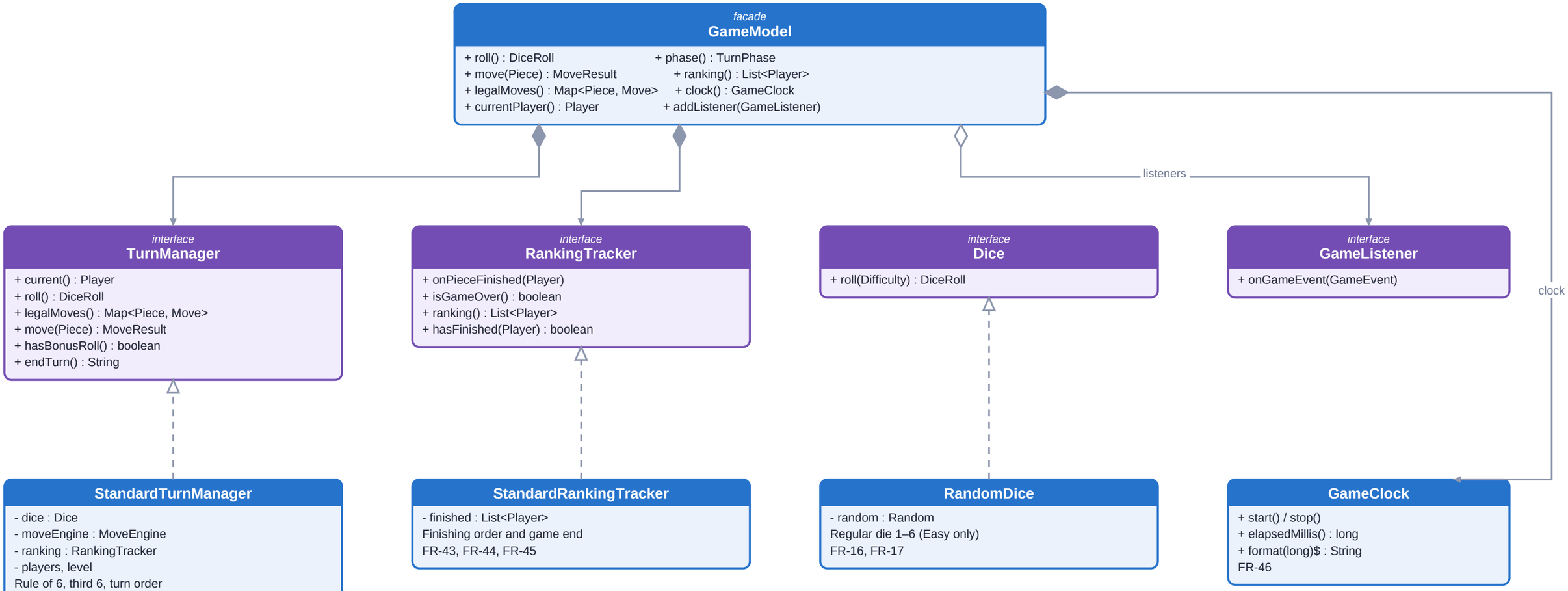
- Model may not import View or Controller, and may not import Swing or AWT.
- View may not import Controller.
- Only Model, View and Controller exist as top-level packages.
- Checked by Iteration1/test/Architecture/MvcDependencyTest.java — the build fails if it is broken.
- The Model never names the View or the Controller. It publishes a GameEvent to whoever registered as a GameListener,
- and the Controller passes the change on to the screens. That is why no arrow above leaves the Model.

Notation



Model — game flow

The GameModel facade and the contracts frozen at checkpoint CP1, each with one Standard implementation



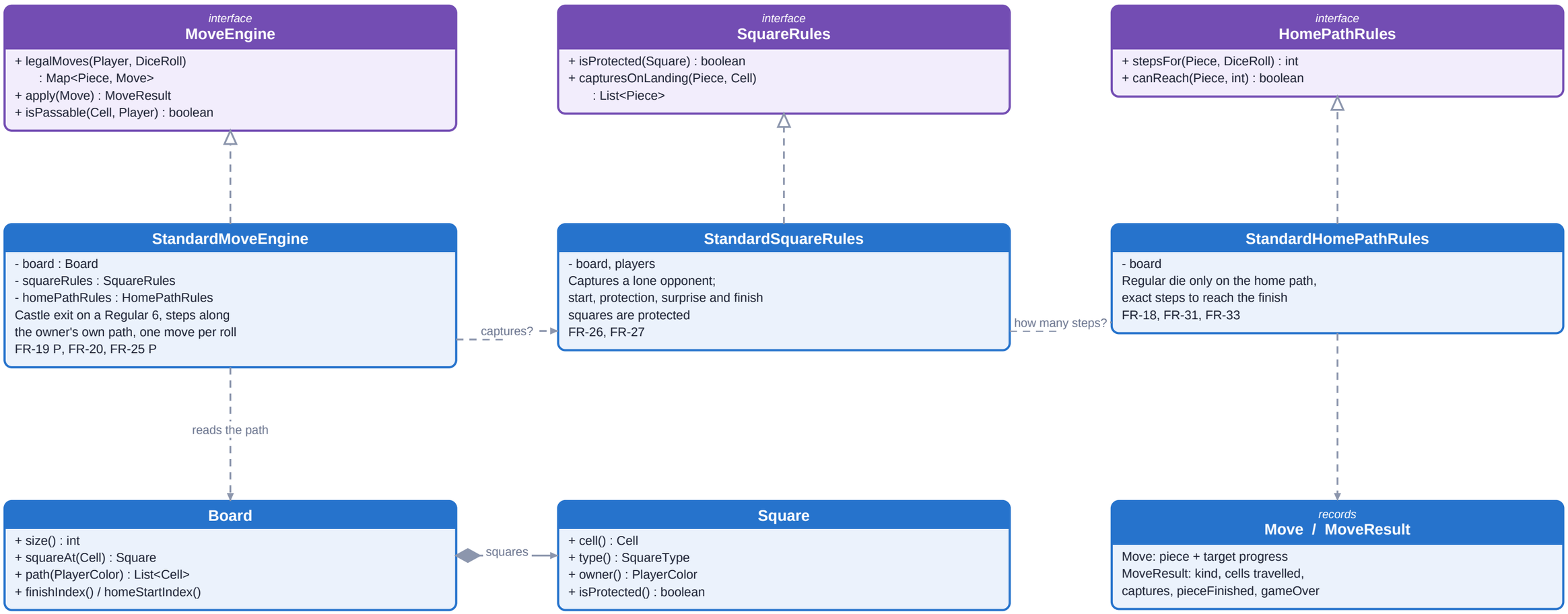
Why interfaces: the six contracts above were frozen at CP1 so the three developers could work in parallel

- Changing one of them needs a change request agreed by all three members (CONTRIBUTING.md).
- MoveEngine, SquareRules and HomePathRules are the other three contracts — page 3.
- GameSettings, Player and Piece, the data a game is built from — page 4.
- StandardTurnManager holds Dice, MoveEngine and RankingTracker as fields (listed in its box above).

Plain data types, drawn where they are used: Cell, DiceRoll, Move, MoveResult, GameEvent, PlayerColor, SquareType, DieType, Direction, Difficulty, TurnPhase.

Model — movement rules

How one legal move is worked out: the engine asks two rule objects and never decides a square rule itself



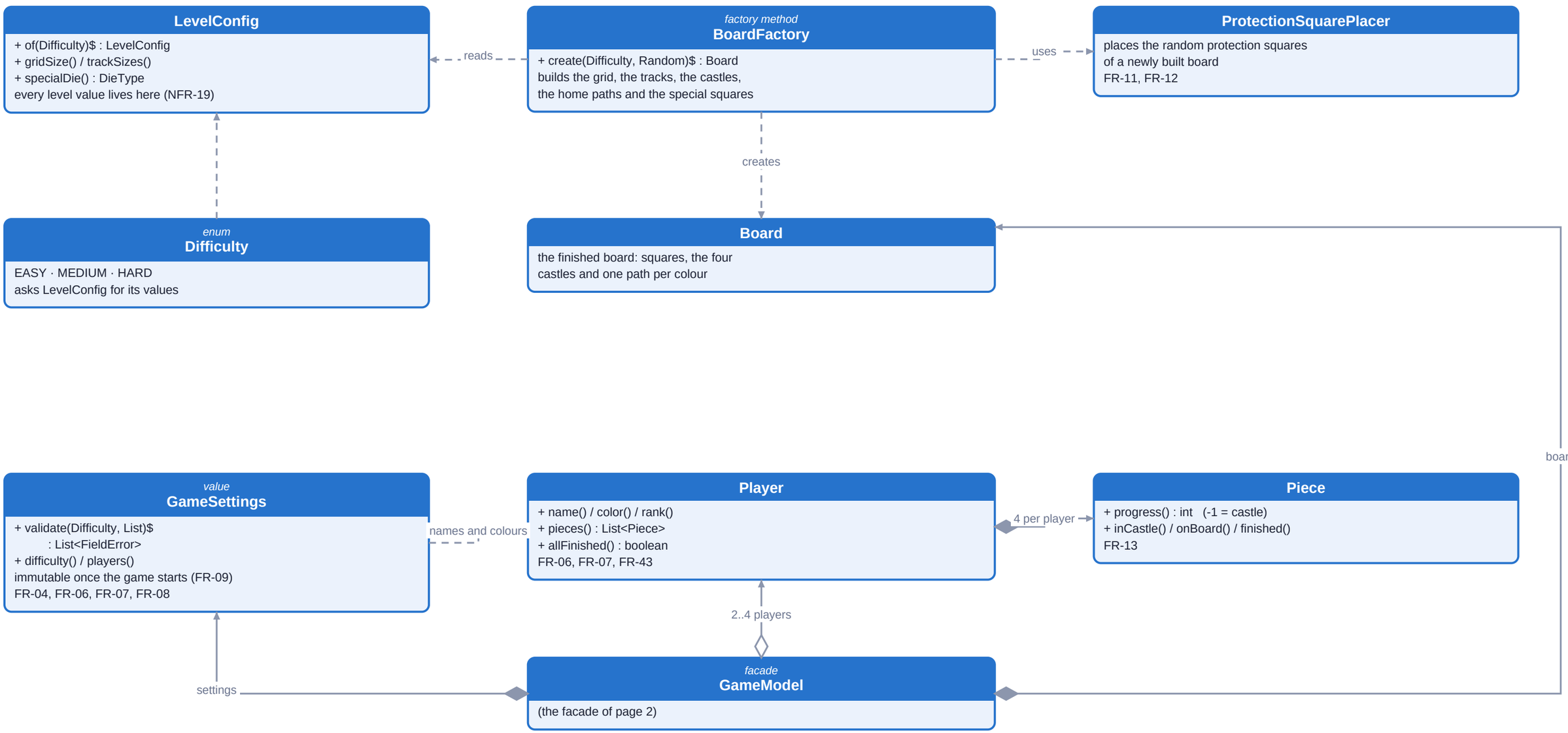
One question, three owners — this split is what let three people work on movement at the same time

- StandardMoveEngine (Task 16, Rakan) decides whether a piece may move and applies the move.
- StandardSquareRules (Task 17, Shafeek) decides capture and protection.
- StandardHomePathRules (Task 18, Shafeek) decides the steps and the exact finish.
- Double Blocks (FR-28..FR-30) arrive in Iteration 2; isPassable() is already the hook for them.

The engine holds the two rule objects behind their interfaces, so either can be replaced without touching the engine.

Model — building a board, and the data of a game

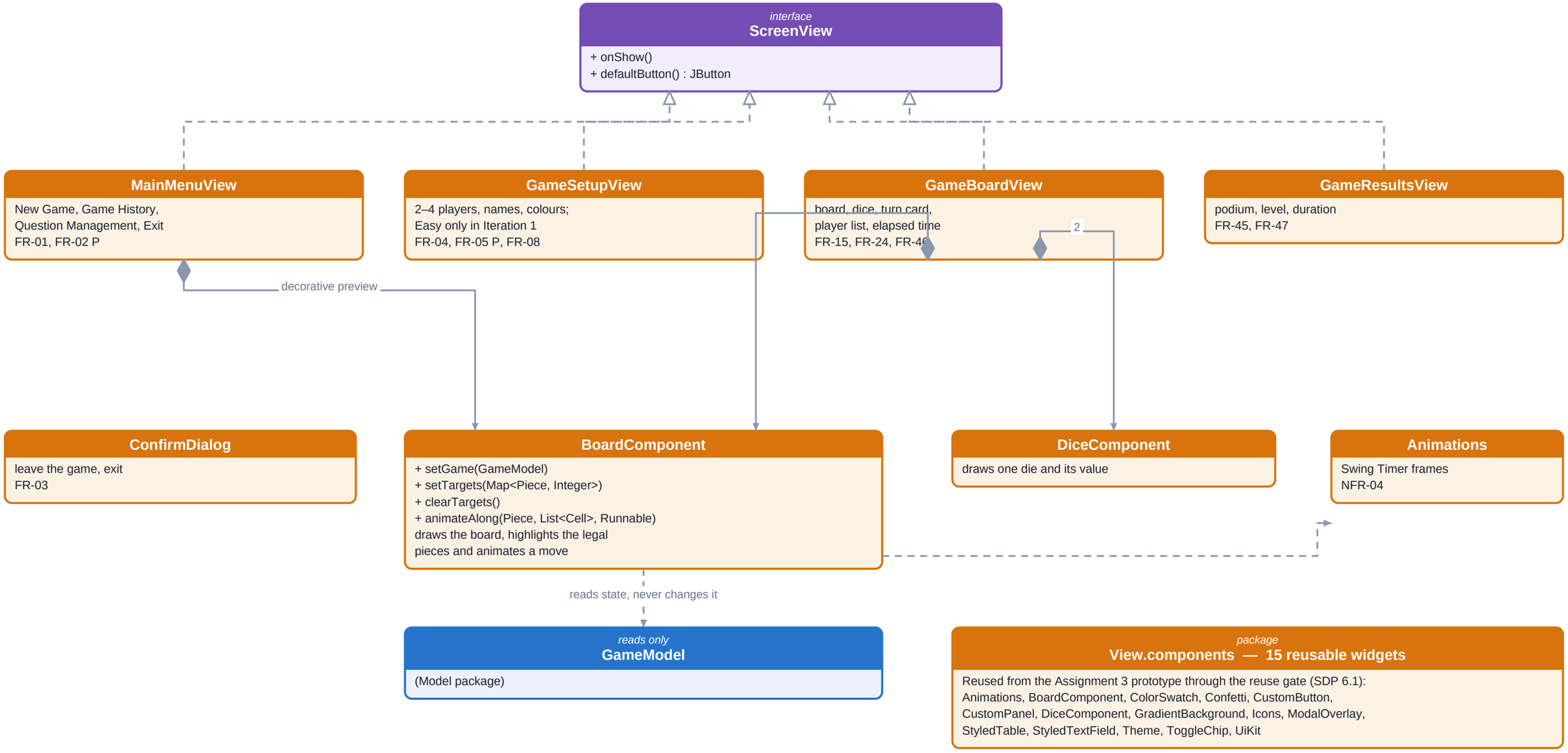
Factory Method (NFR-09) and the settings, players and pieces a game is created from



Factory Method: callers ask BoardFactory for a board of a level and never build one themselves, so Medium and Hard need no change at the call sites (NFR-09, NFR-19).

View — screens and widgets

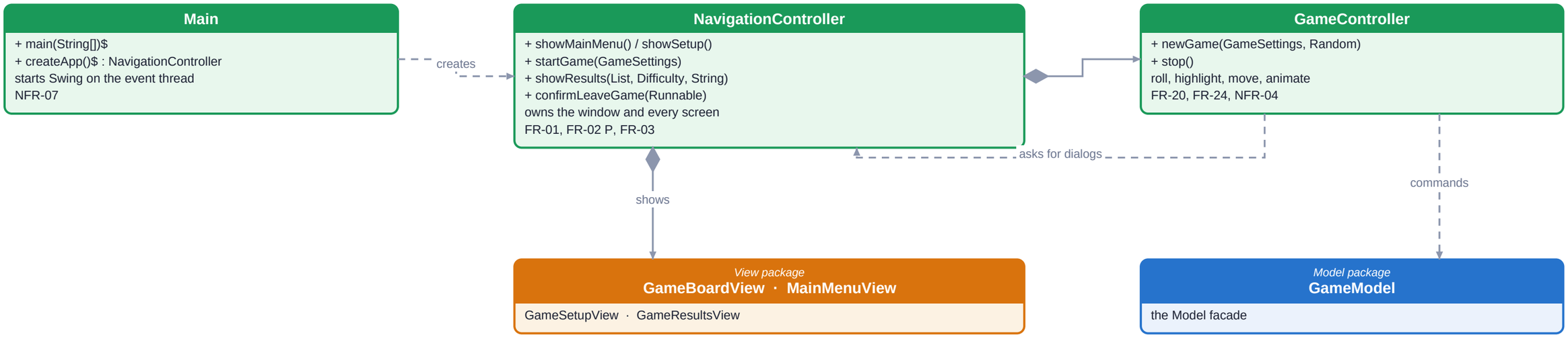
Four screens behind one interface; the board widget draws the Model and never decides a game rule



The View imports Model but never Controller. The Controller registers the listeners that refresh these screens, so the Model stays free of any screen.

Controller — entry point, routing and the game loop

The only package that knows both the screens and the Model



One turn, end to end — the Observer pattern at work (NFR-09)

1. The player presses Roll on GameBoardView.
2. GameController calls GameModel.roll().
3. The Model works the turn out and publishes GameEvent ROLLED — and TURN_PASSED plus TURN_CHANGED when nothing can move (FR-23).
4. GameController, registered as a GameListener, hears the event and asks the Model for the legal moves.
5. It hands them to BoardComponent.setTargets(...), which highlights exactly those pieces (FR-24).
6. The player clicks a highlighted piece. A piece that is not highlighted cannot be clicked at all.
7. GameController calls GameModel.move(piece), animates the result square by square (NFR-04) and clears the highlight.
8. At the end the Model publishes PLAYER_FINISHED and GAME_OVER, and the Controller shows GameResultsView with the ranking and the duration (FR-45, FR-46, FR-47).

Step 3 is the reason no arrow points from the Model to the View or the Controller: the Model publishes an event, it never calls a screen.

Design patterns, and what comes next

NFR-09 · Factory Method, Observer, State and Singleton · the classes planned for Iterations 2 and 3

Factory Method `BoardFactory.create(Difficulty, Random)`	Builds the whole board for a level: grid, tracks, castles, home paths and the random protection squares. All level data comes from LevelConfig.	**Implemented — Iteration 1**
Observer `GameModel + GameListener + GameEvent`	GameModel publishes STATE_CHANGED, ROLLED, MOVED, TURN_PASSED, TURN_CHANGED, PLAYER_FINISHED and GAME_OVER. This is what keeps the Model free of any View import.	**Implemented — Iteration 1**
State `TurnPhase + StandardTurnManager`	AWAITING_ROLL → AWAITING_MOVE → MOVING or AWAITING_ROLL → GAME_OVER. Today an enum, with the transitions written inside the turn manager.	**Partly — becomes State classes in Iteration 2**
Singleton `GameHistory (planned)`	One shared access point to the game-history CSV, so the file cannot be opened twice at the same time.	**Planned — Iteration 3**

Planned for Iterations 2 and 3 — and where each new class attaches

planned · I2
TurnState
+ onRoll() / onMove()
replaces the TurnPhase enum
attaches to StandardTurnManager

planned · I2
QuestionBank / Question
+ nextQuestion(Difficulty)
FR-34..FR-38
attaches to StandardMoveEngine

planned · I2
SurpriseRules
+ outcomeFor(Piece)
FR-39..FR-42
attaches to StandardMoveEngine

planned · I3 · singleton
GameHistory / GameRecord
+ instance()$: GameHistory
+ save(GameRecord) / all()
attaches to GameModel

Iteration 2 also adds: Double Block rules inside StandardMoveEngine, Boost and UltraBoost in RandomDice, the Medium and Hard boards, and SurpriseOutcome.
Iteration 3 also adds: CSV storage for finished games, AdminAuthenticator and the question-management screens.

None of the four classes on this row exists in the code yet. They are drawn dashed, to show only where they will attach.